

Fine-Tuning

Large Language Models

From general-purpose pre-training to specialized, task-ready AI

Prompt Engineering · RAG · Fine-Tuning

Made By Nancy Ahmed

What Is Fine-Tuning?

Fine-tuning takes a **pre-trained model** and continues training it on a smaller, task-specific dataset — adapting its behavior to a particular domain or task.

- The model isn't trained from scratch
- It already holds general knowledge
- We only adapt it to a new, narrower task

Example :THE MEDICAL SCHOOL ANALOGY

Pre-training

Studying all of medicine for six years

Fine-tuning

Specializing in cardiology

Inference

Treating patients



Why Not Train From Scratch?

Pre-training a model from zero is a massive undertaking



Trillions

of tokens of
training data



1,000s

of GPUs
running in parallel



\$Millions

in compute
and infrastructure



Weeks–Months

of continuous
training time



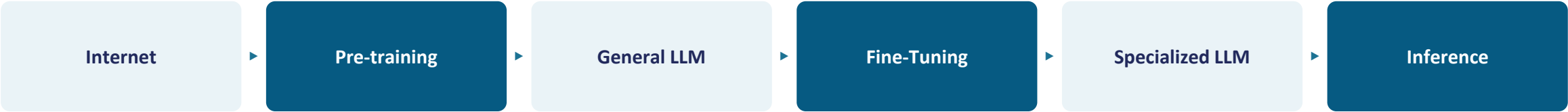
Instead of teaching someone the entire English language again, **you teach them how to write legal contracts.** *That's fine-tuning.*

Fine-tuning reuses the general knowledge already learned — dramatically reducing data, compute, and cost.

Pre-Training vs. Fine-Tuning vs. Inference

	Pre-Training	Fine-Tuning	Inference
Goal	Learn general language	Learn a specific task	Use the model
Data source	Internet-scale data	Small custom dataset	User prompt
Cost	Very expensive	Much cheaper	No training cost
Data volume	Billions–trillions of tokens	Thousands–millions of examples	One request at a time
Frequency	Done once	Done when needed	Happens every use

THE PIPELINE



When Should We Fine-Tune?

Five practical scenarios where fine-tuning earns its cost



Domain Knowledge

Medical chatbots, legal assistants, financial advisors



Style Adaptation

Answer politely, briefly, or in your company's voice



Language Adaptation

E.g., train an English model on Egyptian Arabic



Task Specialization

Translation, SQL generation, summarization, classification



Company-Specific Assistant

Trained on FAQs, manuals, and documentation

Common thread:

Fine-tuning shines when you need consistent, repeatable behavior baked into the model itself.

When Should We NOT Fine-Tune?

The most overlooked question — fine-tuning isn't always the answer



You only need new knowledge

e.g., company documents

→ Use RAG instead



Knowledge changes frequently

weather, news, stock prices, inventory

→ Use RAG, not fine-tuning



Prompt engineering already works

e.g., “Translate this to French”

→ No fine-tuning needed



Your dataset is small

only ~30 examples available

→ Fine-tuning won't help much



Your budget is limited

fine-tuning still needs GPU time

→ Prompting may be enough

Three Ways to Adapt a Model

Why isn't prompting always enough?



Prompt Engineering

Prompt



LLM



Answer

EXAMPLE

"Summarize this in 3 sentences."

- ✓ Fast, free, easy
- ✓ No GPU required
- ✗ Doesn't change the model
- ✗ Must repeat every time



RAG

Question



Retriever



LLM

EXAMPLE

"What's our refund policy?" → retrieves refund_policy.pdf

- ✓ Always up to date
- ✓ No retraining needed
- ✗ Doesn't change writing style
- ✗ Depends on retrieval quality



Fine-Tuning

Dataset



Training



Updated Model

EXAMPLE

Teach the model to always reply in JSON

- ✓ Teaches behavior & format
- ✓ Deep domain adaptation
- ✗ Requires training data
- ✗ Higher upfront cost

Prompt Engineering vs. RAG vs. Fine-Tuning

One of the most useful comparisons in this session

Feature	Prompt Eng.	RAG	Fine-Tuning
Changes model weights	—	—	✓
Needs training	—	—	✓
Uses external documents	—	✓	—
Learns new behavior	—	—	✓
Uses latest knowledge	—	✓	—
Cheapest option	✓	✓	—
Best for style adaptation	—	—	✓
Best for dynamic knowledge	—	✓	—

Checkmarks show relative strength — not exclusivity. Many real systems combine two or more approaches.

Real-World Examples

Match the scenario to the best solution



Customer support with a changing product catalog

Knowledge changes frequently

RAG



Medical report summarizer

Needs consistent domain behavior & style

Fine-Tuning



Translate English to French

The base model already knows the task

Prompt Engineering



Internal chatbot using policy documents

Docs change; tuning can add style

RAG (+ Fine-Tuning)



Generate SQL in your company's format

Requires a consistent output format

Fine-Tuning

KEY TAKEAWAY

Choose the simplest tool that solves the problem.



Prompt engineering when the model already knows how



RAG when knowledge is external or changes often



Fine-tuning when you need consistent behavior, style, or format

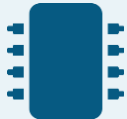
FINE-TUNING LLMS · PART 2

PEFT, Data & Tokenization

How to fine-tune efficiently, prepare clean data, and turn text into tokens

[LoRA](#) · [QLoRA](#) · [Dataset Prep](#) · [Tokenization](#)

Why Not Update All the Weights?



Suppose you want to fine-tune a 7B parameter model. Do you really need to update all 7 billion parameters? **No.**

Full fine-tuning updates every parameter — costly in GPU memory and storage. Research shows updating only a **small subset** can reach similar performance. **PEFT** keeps the original weights frozen and trains only a small number of new parameters.

	Full Fine-Tuning	PEFT
Parameters updated	All weights	A few parameters
GPU memory	High	Low
Checkpoint size	Large	Small adapters
Training speed	Slower	Faster

LoRA: The Core Idea

Instead of replacing the original weight matrix W , LoRA learns a small update on top of it:

$$W' = W + BA$$

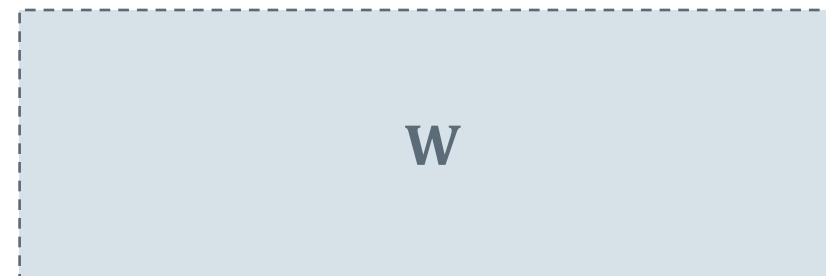
W

Original pretrained weights — frozen, untouched

A and B

Small trainable matrices — only these get updated

FROZEN VS. TRAINABLE



frozen · not updated

+



small · trainable

Rank (r): How Much Can Change?

Rank determines how much the model is allowed to change

SMALL RANK (e.g. 4 or 8)

- Fewer trainable parameters
- Less memory required
- Less flexibility to adapt

LARGE RANK (e.g. 64 or 128)

- More trainable parameters
- Higher memory usage
- Greater flexibility to adapt



THE DOCUMENT-EDITING ANALOGY

Rank = 4 → only a few sentences can change. **Rank = 64** → many paragraphs can change.

Q = QUANTIZED

QLoRA: Quantized LoRA



PRECISION & MEMORY TRADE-OFF

Precision	Bits	Memory	Trade-off
FP32	32 bits	Highest	Full precision
FP16	16 bits	High	Minimal loss
INT8	8 bits	Low	Slight accuracy trade-off
INT4	4 bits	Very low	Used by QLoRA



Why QLoRA took off

Before: needed multiple expensive GPUs. After QLoRA: fine-tune a 7B model on a single modern GPU.

The lower the precision, the less memory and faster the compute — at a small accuracy cost.

Other PEFT Methods at a Glance



AdaLoRA

Automatically adjusts LoRA rank during training, allocating more capacity where it's most useful



IA³

Learns scaling vectors for activations instead of adding matrices — very lightweight



Prefix Tuning

Adds trainable prefix vectors before the input sequence; model weights stay frozen



Prompt Tuning

Learns continuous (soft) prompt embeddings instead of updating model weights



P-Tuning v2

An improved Prompt Tuning that works well across model sizes and tasks

PEFT Methods Compared

Method	Updates Weights?	Memory	Popularity
Full FT	All	High	★★★★☆
LoRA	Small adapters	Low	★★★★★
QLoRA	Quantized + LoRA	Very Low	★★★★★
IA ³	Scaling vectors	Very Low	★★★★☆
Prompt Tuning	Soft prompts	Very Low	★★★☆☆
Prefix Tuning	Prefix embeddings	Very Low	★★★☆☆

LoRA and QLoRA dominate in practice — the best balance of low memory cost and strong performance.

What Makes a Good Dataset?

A model can only learn from the quality of its data

✓ GOOD DATASET

- Correct
- Consistent
- Diverse
- Relevant to the task
- High quality

Q: What is machine learning?

A: Machine learning is a field of AI where models learn patterns from data.

✗ BAD DATASET

- Wrong answers
- Duplicate samples
- Mixed languages (unintentional)
- Missing labels
- Poor formatting
- Contradictory examples

DATASET PREPARATION

Cleaning, Deduplication & Splitting

DATA CLEANING

-  Remove duplicates & empty rows
-  Fix spelling mistakes where appropriate
-  Normalize whitespace
-  Remove corrupted samples
-  Filter offensive or irrelevant content



If the same sample appears 1,000 times, the model may overfit to it.

DATASET SPLITTING



Train — The data the model actually learns from

Validation — Used to tune settings and catch overfitting early

Test — Held out entirely — measures final real-world performance

Dataset Formats: Instruction & Chat



Instruction Tuning

instruction — what to do

input — optional context

output — expected answer

```
{
  "instruction": "Translate to Arabic",
  "input": "Good morning",
  "output": "صباح الخير"
}
```



Chat Format

system — defines behavior

user — the question

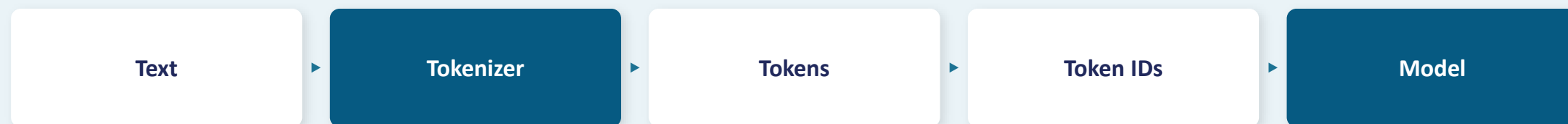
assistant — the response

```
[
  {"role": "system", "content": "You are helpful."},
  {"role": "user", "content": "What is AI?"},
  {"role": "assistant", "content": "AI is..."}
]
```

Most modern chat models — Llama, Qwen, Gemma, Mistral — are fine-tuned using conversational datasets like this.

From Text to Numbers

Can a neural network understand text directly? **No.** It understands numbers, not words.



`"Hello world"` → `["Hello", "world"]` → `[15496, 995]`

WHY TOKENIZATION MATTERS



Converts text into numbers



Defines the model's vocabulary



Affects usable context length



Impacts memory usage & cost

Sequence Length, Truncation & Padding

Every model has a maximum sequence length:

512

2048

4096

8192

128K



Truncation

Cut off tokens beyond the model's maximum sequence length.

```
Limit = 8 · Input = 12 tokens → keep first 8,  
discard last 4
```



Padding

Adds special tokens so every sequence in a batch matches in length.

```
"I love AI" (4 tok) · "Hello" → Hello <PAD> <PAD> <PAD>
```



Attention Mask — padding tokens are ignored by the model via an attention mask, so they don't affect learning.

KEY TAKEAWAY

The Complete Preprocessing Pipeline



PEFT (LoRA / QLoRA) makes fine-tuning fast and affordable



Clean, well-formatted data matters more than data volume



Tokenization, padding, and truncation shape what the model actually sees

MODEL FINE-TUNING

The Training Pipeline

From raw dataset to fine-tuned model: tokenization, batching, forward passes, loss, backpropagation, optimizers, and the hyperparameters that steer it all.



Dataset



Tokenizer



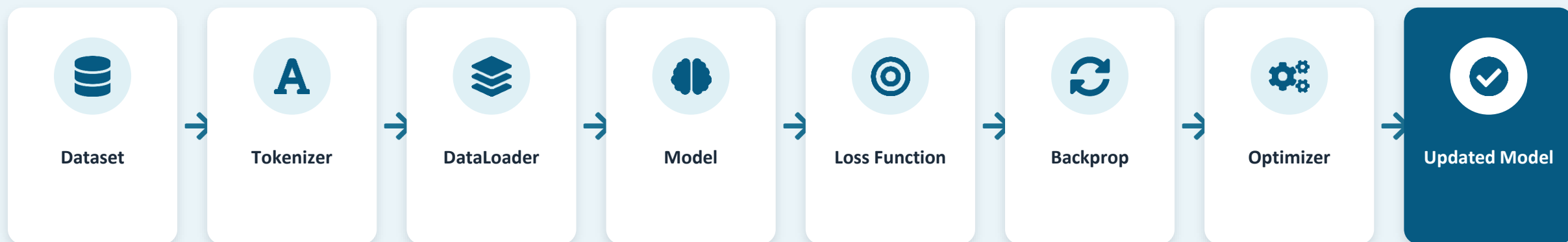
Model



Optimizer

The Complete Training Pipeline

Eight stages carry raw data through the model and back out again as improved parameters.



Each step has one job:

move examples in → turn text into numbers → batch them efficiently → predict the next token → measure error → attribute blame → update weights.

Dataset & Tokenizer



Dataset

The collection of training examples — the source of everything the model learns during fine-tuning.

Instruction:

Translate to Arabic

Input: Hello

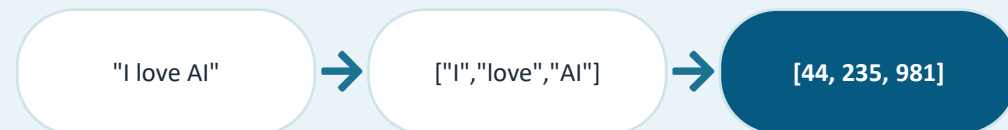
Output: مرحبا

Formats vary — instruction/input/output or user/assistant turns — but the goal is always the same: teach by example.



Tokenizer

Models can't understand text. The tokenizer converts text into numbers the model can process.



Key concepts

- Vocabulary — the fixed set of known tokens
- Token IDs — numeric index for each token
- Padding — fills short sequences to equal length
- Truncation — cuts sequences that run too long

STEP 3

DataLoader — Feeding Data in Batches

The DataLoader feeds the dataset to the model in small batches instead of all at once.

1,000

Examples in dataset

16

Batch size

63

Batches per epoch



Dataset sends batches sequentially until it's exhausted

Batch 1
(16 ex.)

Batch 2
(16 ex.)

Batch 3
(16 ex.)

...

Batch 63
(16 ex.)

Why batch instead of all at once?

Fits GPU memory

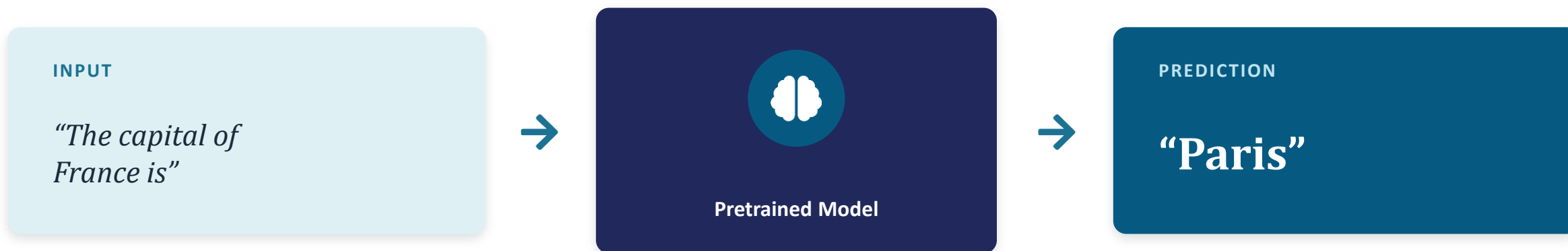
Faster training

Better optimization

STEP 4

The Model Makes a Prediction

Token IDs enter the pretrained model, which predicts the next token — or the desired output, depending on the task.



At this point, nothing has been learned yet.

This is called the forward pass: the model receives input and produces a prediction. Learning only happens later, once that prediction is compared against the correct answer and the error flows backward through the network.

Loss Function — Measuring Error

The model compares its prediction to the correct answer. That comparison produces a single number: the loss. Smaller loss means a better prediction.



Confident & correct

Target: "dog"



Small loss



Uncertain & wrong

Target: "dog"



Large loss

Backpropagation & the Optimizer



Backpropagation

This is where learning actually happens.

The model asks “what caused my mistake?” and calculates how much each trainable parameter contributed to that error — no manual math required to understand the idea.

Error signal flows backward

Output layer → Hidden layers → Input layer

Each parameter gets a gradient: a signal for how it should change.



Optimizer

The optimizer takes the gradients from backprop and uses them to update the trainable parameters.

Old weights



Optimizer



New weights

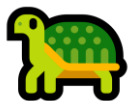
Common optimizers

AdamW — most common for LLMs

SGD — older, still used

Learning Rate — the Most Important Setting

Hyperparameters are settings we choose before training — unlike model weights, which are learned. Learning rate controls how much the model changes after each update.



Too Small

Slow learning — training crawls, wastes compute, may stall before converging.



Good

Steady, reliable improvement toward the best solution.



Too Large

Overshoots the best solution — loss can bounce or diverge.



Typical learning rate values for LLM fine-tuning

$1e-5$

$2e-5$

$5e-5$

Epochs & Batch Size



Epochs

One epoch = one complete pass through the entire dataset.

Epoch 1

Model sees all 1,000 examples

Epoch 2

Model sees them again

- Too few epochs → underfitting
- Too many epochs → overfitting



Batch Size

How many examples are processed together in one update.



Batch size = 8 → 8 samples processed → 1 update

Larger batches

- Faster
- More memory

Smaller batches

- Slower
- Less memory

Gradient Accumulation

A technique for simulating a larger batch size than your GPU memory allows.

The problem: GPU only fits a batch of 2, but you want the stability of a batch of 8.



Effective batch size = 8

Gradients from each mini-batch are summed before a single optimizer step — giving the stability of a large batch within a small memory budget.

Weight Decay, Warmup & Sequence Length



Weight Decay

Keeps weights from becoming unnecessarily large — preventing the model from simply memorizing the training data. Reduces overfitting.

Large weights

↓ regularized ↓

Balanced weights



Warmup Steps

Learning rate ramps 0 → small → medium → target instead of starting at full strength. Produces stable training and prevents early divergence.



Max Sequence Length

Maximum number of tokens the model accepts per example. Longer = more memory & compute; shorter = faster but may lose information.

e.g. 2,048 / 4,096 / 8,192 tokens

Evaluation Steps & Checkpoint Saving



Evaluation Steps

Don't wait until the end — evaluate on a validation set every N training steps.

Train → step N → Evaluate → step 2N → Evaluate → ...

Benefits

- Detect overfitting early
- Monitor progress objectively



Checkpoint Saving

Instead of saving only the final model, save periodically throughout training.

Checkpoint 500

Checkpoint 1000

Checkpoint 1500

Benefits

- Resume interrupted training
- Compare models across steps
- Recover the best checkpoint

Cross Entropy & the Training Curve



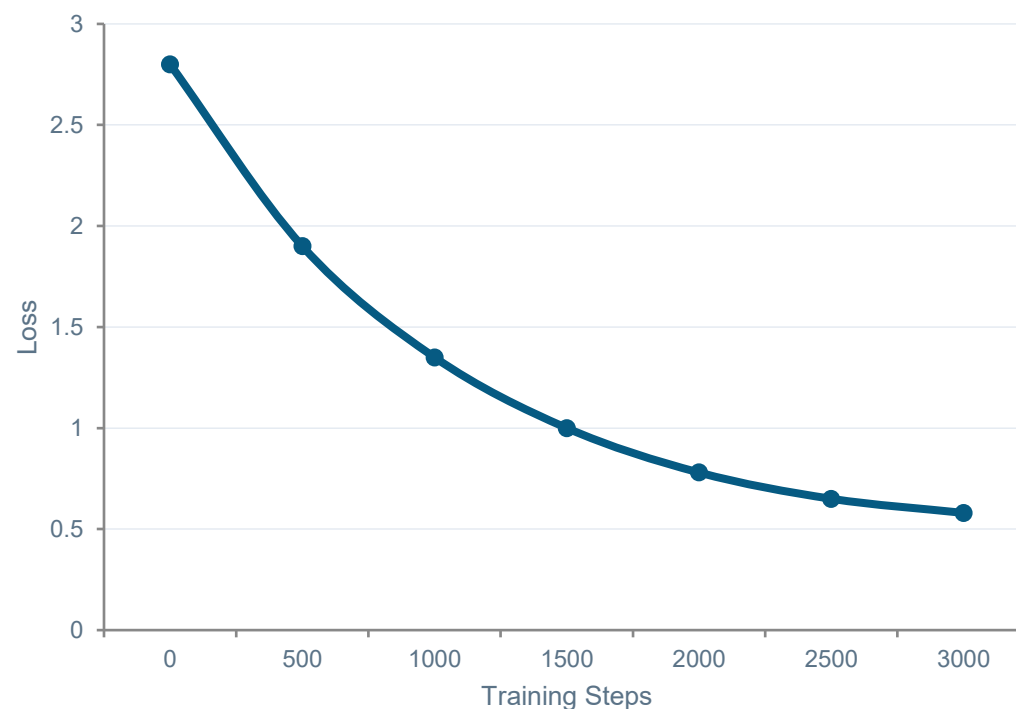
Cross Entropy

The most common loss function for language models.

It compares predicted probabilities with the correct answer — rewarding high confidence on the correct class and penalizing incorrect predictions.

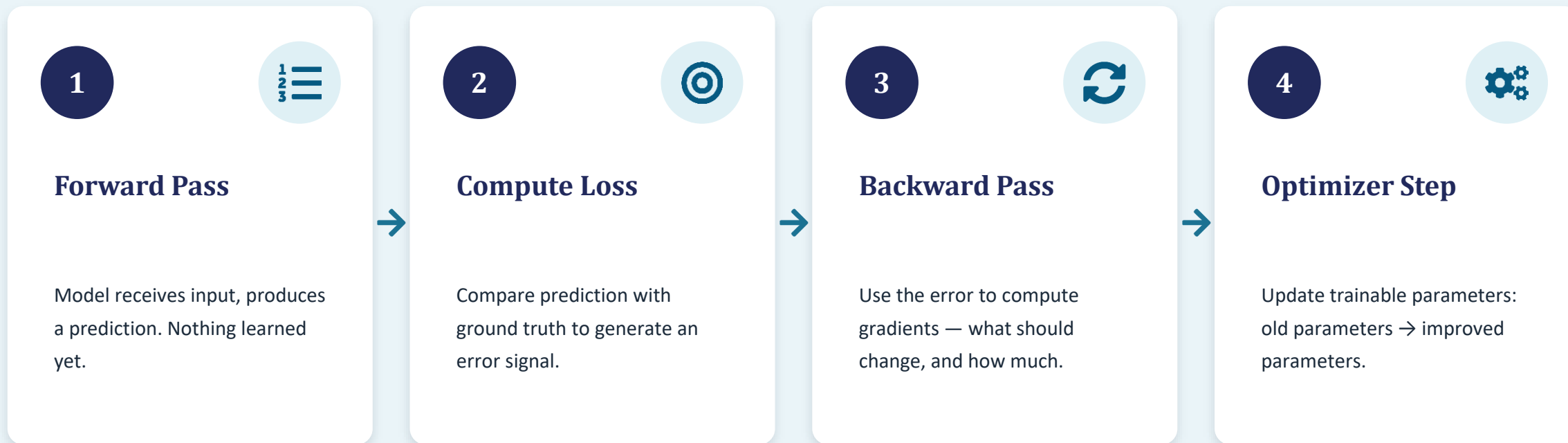
Good training means loss decreases over time — that's the single clearest signal that fine-tuning is working.

Training Loss Curve



One Complete Training Iteration

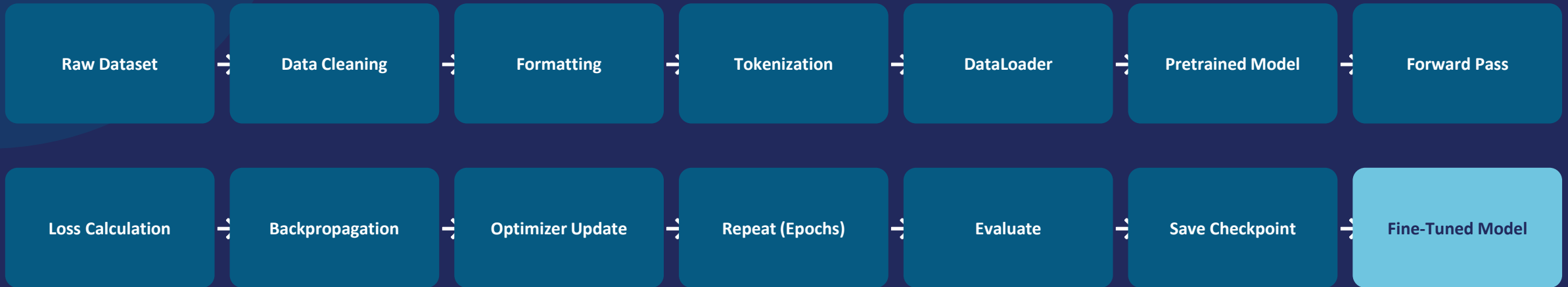
Training isn't a single cycle — it's this loop, repeated thousands of times, until loss decreases and the model improves.



Repeat thousands of times — forward → loss → backward → update → forward → ...

SUMMARY

The Complete Fine-Tuning Workflow



From data to a deployed, fine-tuned model — one pipeline, repeated until the loss curve tells you it's ready.



MODEL FINE-TUNING SERIES

Evaluation, Overfitting & the Fine-Tuning Workflow

Metrics that matter, why models overfit, and how the Hugging Face stack turns theory into a working adapter.

11 · Evaluation

12 · Overfitting

13 · HF Ecosystem

14 · Workflow

11 Evaluation — Classification Metrics

Metrics depend on the task. For classification, these four form the core toolkit.



Accuracy

Share of all predictions the model got exactly right.



Precision

Of predicted positives, how many were actually correct.



Recall

Of actual positives, how many the model successfully found.



F1 Score

Harmonic mean of precision and recall — a single balance point.

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) \div (\text{Precision} + \text{Recall})$$

11 Evaluation — Generation Metrics

Open-ended text output needs different yardsticks than a fixed label set.



BLEU

N-gram precision overlap with reference text; standard for translation.



ROUGE

N-gram / sequence recall overlap; standard for summarization.



BERTScore

Embedding-based similarity that captures meaning, not just wording.



Perplexity

How well the model predicts held-out text — lower is more confident.



Human Evaluation

Automatic metrics miss fluency, factuality, and tone — human judgment remains essential, especially for generative and open-ended tasks.

12 Preventing Overfitting



Early Stopping

Halt training once validation metric stops improving.



Regularization

Dropout, weight decay, and label smoothing constrain complexity.



More Data

Larger, diverse datasets reduce memorization risk.

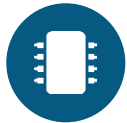


Data Augmentation

Synthetic variations expose the model to broader patterns.

13 The Hugging Face Ecosystem

Seven libraries, each covering one stage of the model lifecycle.



Transformers

Pretrained model architectures and weights, ready to load and fine-tune.



Datasets

Fast, memory-efficient loading and processing for datasets of any size.



Tokenizers

High-speed tokenization that converts raw text into model-ready inputs.



PEFT

Parameter-efficient fine-tuning — LoRA and friends — to adapt models cheaply.



TRL

Reinforcement learning tools for aligning models via reward or preference signals.



Accelerate

Runs the same code seamlessly across CPU, GPU, and multi-device setups.



Evaluate

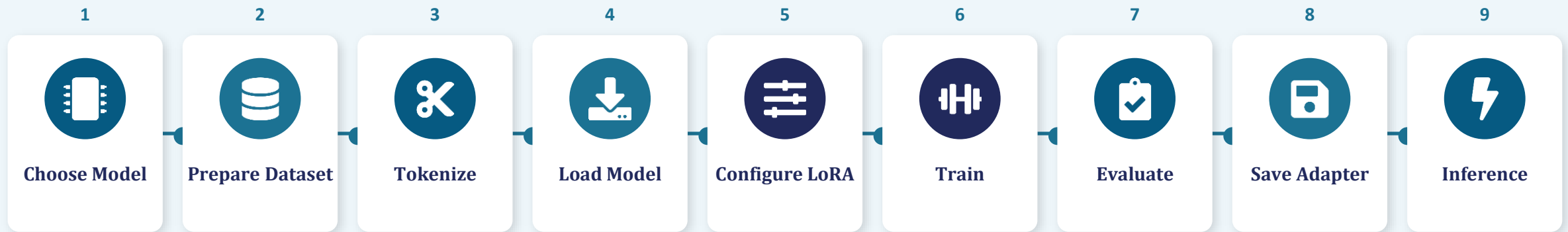
A unified interface for computing standard metrics like BLEU and F1.

One ecosystem — model to metric.

Each library plugs into the next, forming a single connected workflow.

14 Fine-Tuning Workflow

Nine steps from a pretrained checkpoint to a working, task-tuned model.



Where the real fine-tuning happens

After tokenized data meets the loaded base model, LoRA adapters are configured to update only a small set of low-rank weights — keeping training fast and memory-light.



From adapter to answer

The saved adapter is merged or loaded alongside the base model at inference time, delivering task-specific behavior without retraining the full network.

MODEL FINE-TUNING

Code, Applications & Real-World Tradeoffs

From `trainer.train()` to production: writing the fine-tuning code, where it's used in industry, fine-tuning vs. RAG, and the challenges you'll actually run into.



Code



Applications



RAG



Challenges

Step 1 — Import the Libraries

Now we map everything we've learned onto actual code. Each import brings in one piece of the pipeline.

```
from transformers import AutoTokenizer
from transformers import
AutoModelForCausalLM
from peft import LoraConfig
from trl import SFTTrainer
```

transformers	Load pretrained models & tokenizers
peft	Configure LoRA / QLoRA adapters
trl	Fine-tuning trainer for chat & instruction models



Check your understanding

“Which Hugging Face library does each import come from?”

This reinforces the previous section — tie each library back to the pipeline stage it powers.

Step 2 & 3 — Load Tokenizer & Model



Load the Tokenizer

```
tokenizer = AutoTokenizer.from_pretrained(...)
```

- Downloads the tokenizer
- Converts text into tokens
- Uses the tokenizer that matches the model

"Hello world"



["Hello", "world"]



[15496, 995]



Load the Model

```
model =  
AutoModelForCausalLM.from_pretrained(...)
```

- Downloads the pretrained model
- Loads the pretrained weights
- At this point, nothing has been trained yet

If using QLoRA

- Load the model in 4-bit
- Freeze the original weights

Step 4 — Configure LoRA

```
config = LoraConfig(  
    r=16, lora_alpha=32, lora_dropout=0.05,  
    target_modules=[...]  
)
```



Rank (r)

How much the model is allowed to adapt.
Small rank → less memory. Large rank → more flexibility.



lora_alpha

A scaling factor — controls the strength of the LoRA updates.



lora_dropout

Regularization that helps reduce overfitting.



target_modules

Tells LoRA where to insert adapters, e.g. q_proj, k_proj, v_proj, o_proj — specific layers in the attention mechanism.

Step 5 & 6 — Trainer & Training



Create the Trainer

```
trainer = SFTTrainer(...)
```

This is where every previous concept comes together:

- Model
- Tokenizer
- Dataset
- Training arguments
- PEFT configuration



`trainer.train()`

One line executes the entire training loop already covered:

Load batch

Tokenize

Forward pass

Loss

Backward pass

Optimizer

Update LoRA adapters

Repeat

Step 7 & 8 — Save & Run Inference



Save the Model

```
trainer.save_model(...)
```

With LoRA, only the adapter is saved — not the full model.

~15 GB

Full model save

100–300 MB

LoRA adapter only

A tiny adapter file instead of a full multi-GB checkpoint — fast to store, share, and swap.



Inference

```
model.generate(...)
```

Training is finished — now we're using the model.

Prompt

Tokenizer

Model

Generated Tokens

Decoded Text

The Final Code Flow



What students should notice

A single call — `trainer.train()` — executes the entire training loop you've already learned: batching, tokenizing, forward pass, loss, backward pass, and optimizer updates, repeated automatically until training is complete.

Why Companies Invest in Fine-Tuning



Medical Assistant

Dataset:

Medical Q&A, clinical guidelines

Result:

Medical chatbot



Legal Assistant

Dataset:

Laws, contracts, court rulings

Result:

Legal document assistant



Customer Support

Dataset:

FAQs, tickets, chat history

Result:

Company-specific chatbot



Financial Assistant

Dataset:

Banking policies, financial reports

Result:

Risk analysis, investment support



Code Generation

Dataset:

Python, Java, SQL, GitHub repos

Result:

Completion, bug fixing, docs



Arabic Chatbot

Dataset:

Arabic conversations, Egyptian & Gulf dialect

Result:

Better Arabic than the base model

Task-Level Fine-Tuning Examples



Sentiment Analysis

Classify text as Positive, Negative, or Neutral. Example: product reviews.

Positive

Negative

Neutral



Named Entity Recognition

Extract Person, Organization, Location, Date from text.

Microsoft hired **Ahmed** in **Cairo**



SQL Generation

Input:

"Show all employees hired after 2023."

```
SELECT * FROM employees WHERE ...
```



Document Summarization

Condense long documents into concise summaries.

Research papers

Legal contracts

News

Medical reports

The Most Common GenAI Interview Question



Fine-Tuning

Changes the model — learns behavior, style, and format.



RAG

Leaves the model unchanged — adds external knowledge at query time.

	Fine-Tuning	RAG
Changes weights	✔ Yes	✗ No
Learns behavior	✔ Yes	✗ No
Uses external documents	✗ No	✔ Yes
Dynamic knowledge	✗ Poor	✔ Strong
Compute cost	✔ More expensive	✗ Cheaper per query
Updatable	Harder — retrain needed	✔ Easy — update the index

Choosing — and Combining — the Right Approach

Real-World Examples

Company manuals

RAG

Medical writing style

Fine-Tuning

Latest news

RAG

Customer service tone

Fine-Tuning

Internal docs + company tone

RAG + Fine-Tuning

Most production systems combine them

Company Documents

Retriever

LLM

Fine-Tuned Behavior

RAG provides **knowledge** • Fine-tuning provides **behavior**

Model Behavior & Resource Constraints



Catastrophic Forgetting

The model forgets previous knowledge while learning new data.

LoRA

Mixed datasets

Smaller learning rates



Hallucinations

The model generates incorrect information confidently.

Better datasets

RAG

Human evaluation

Fact checking



Limited GPU Memory

Large models require lots of VRAM.

LoRA

QLoRA

Gradient accumulation

Mixed precision



Poor Dataset Quality

Garbage in → garbage out.

Clean

Validate

Deduplicate

Data & Fit Problems



Data Imbalance

e.g. 95% Positive vs. 5% Negative — the model becomes biased.

Resampling

Balanced datasets

Weighted loss



Long Context

Input exceeds the model's context window.

Chunking

Sliding windows

Long-context models

RAG



Overfitting

Training loss keeps falling while validation loss rises.

Early stopping

Weight decay

More data



Underfitting

The model doesn't learn enough from the data.

More epochs

Better learning rate

Better dataset

RECAP

From Code to Production



Eight lines of code carry the pipeline from tokenizer to a saved LoRA adapter



Fine-tuning powers medical, legal, support, financial, code, and language use cases



RAG adds knowledge; fine-tuning adds behavior — most systems need both



Every challenge — forgetting, hallucination, memory, data quality — has a known mitigation